

PROPUESTA TÉCNICA

Documento N.º 001 · Semana 5 · Serie de seguimiento del proyecto

TRADUCTOR DE LENGUAJE DE SEÑAS A VOZ

MEDIANTE SENSORES FLEX Y MICROCONTROLADOR ESP32

Reconocimiento de gestos manuales aplicado a comunicación asistida

COORDINADOR	Dr. Camilo Caraveo Mena
PRESENTADO POR	Andrés Ivan Abundiz Rivera · Ayudante
SEMESTRE / CICLO	Cuarto semestre · [2026-2]
ESTADO	Preliminar — definición de hardware y modelo de I.A.
FECHA	[14 de septiembre de 2026]

Tabla de contenido

1. Resumen del proyecto	3
1.1 Objetivo general.....	3
1.2 Objetivos específicos	3
2. Comparativa de plataformas ESP32	4
2.1 Problemas a solucionar	5
3. Diagrama de conexiones.....	6
3.1 Conexión individual de cada sensor flex.....	6
4. Propuestas de implementación física en el guante	7
5. De los sensores al modelo: pipeline de reconocimiento	8
5.1. Adquisición y preprocesamiento	8
5.2. Ventana de decisión.....	8
5.3. Modelo de clasificación.....	8
5.4. Datos de entrenamiento (preliminar)	8
6. Lista de materiales preliminar	9
7. Próximos pasos	9

1. Resumen del proyecto

Este proyecto propone el desarrollo de un guante instrumentado capaz de traducir el lenguaje de señas a voz en tiempo real. Cada dedo incorpora un sensor flex que mide el ángulo de flexión; después, estas lecturas se procesan en un microcontrolador ESP32, que reconoce patrones asociados a gestos o letras y produce una salida hablada (por altavoz o mediante una aplicación móvil vía Bluetooth, discutidos en la sección 2.1.1).

1.1 Objetivo general

Diseñar e implementar un sistema portátil basado en ESP32 y sensores flex que traduzca gestos de la mano correspondientes al lenguaje de señas a salida de voz, como apoyo a la comunicación asistida.

1.2 Objetivos específicos

- Seleccionar la variante de ESP32 (regular, mini o tipo reloj con sensor de muñeca) más adecuada para la integración en el guante.
- Diseñar el circuito de acondicionamiento de señal (con resistencias) para cada sensor flex.
- Implementar la lectura simultánea de los 5 sensores flex, el sensor IMU y su mapeo a un conjunto inicial de letras o gestos.
- Definir el canal de salida (voz local, aplicación móvil por Bluetooth, o ambos).
- Evaluar ergonomía, consumo energético y autonomía de al menos dos configuraciones físicas del guante.

2. Comparativa de plataformas ESP32

Se evalúan tres variantes de ESP32 como posible núcleo del sistema. Los valores son referenciales y deben confirmarse con la hoja de datos de la placa específica que se adquiera, ya que existen múltiples fabricantes para cada formato.

Criterio	ESP32 DevKit (regular)	ESP32 Mini (p. ej. C3 SuperMini)	ESP32-S3 tipo reloj (sensor de muñeca)
Procesador	Xtensa LX6 doble núcleo, 240 MHz, 520 KB SRAM	Según el chip real de la placa: si es un WROOM-32 miniaturizado, igual que el regular; si es un ESP32-C3, RISC-V de un solo núcleo, sin extensiones vectoriales	ESP32-S3R8: Xtensa LX7 doble núcleo, 240 MHz, 8 MB PSRAM, con instrucciones vectoriales para IA (SIMD)
Factor de forma	~55 x 28 mm, placa plana	~22 x 18 mm, muy compacta	Carcasa de reloj de pulsera, con pantalla
GPIO / entradas ADC	~25 GPIO, 8 canales ADC	~11-13 GPIO, 5-6 canales ADC	Pocos GPIO libres (ocupados por pantalla/batería)
Sensor de movimiento (IMU)	No integrado (requiere MPU6050 externo)	No integrado	Sí, integrado de fábrica en la muñeca
Conectividad	Wi-Fi + Bluetooth clásico/BLE	Wi-Fi + BLE (sin BT clásico en C3)	Wi-Fi 2.4 GHz 802.11 b/g/n + Bluetooth 5 (LE), antena integrada
Consumo / autonomía	Consumo mayor, picos ~500 mA en TX	Consumo similar pero placa más chica	Mayor por la pantalla; autonomía más ajustada
Calidad del ADC	Conocido por ser algo ruidoso/no lineal cerca de los extremos; conviene calibrar (esp_adc_cal)	Depende del chip real	Linealidad mejorada respecto al ESP32 original
Integración en el guante	Requiere carcasa en el dorso de la mano	Se oculta más fácil, pero soldadura fina	Se usa en la muñeca; sensores por cable plano a los dedos
Costo aproximado	USD 4-7	USD 3-5	USD 15-35 según modelo
Recomendación	Ideal para el prototipo de banco / primeras pruebas	Buena opción para una versión compacta final	Opción más ambiciosa; mayor riesgo técnico y de pines

Tabla 1. Comparación entre variantes de ESP32 para este proyecto.

2.1 Problemas a solucionar

Antes de avanzar con la integración física, quedan dos decisiones abiertas: cómo se resuelve la salida de audio y cómo se ejecuta el modelo de reconocimiento dentro del microcontrolador. Ambas se documentan aquí como definición preliminar, sujeta a confirmarse.

2.1.1. Salida de audio

Ninguna de las tres variantes de ESP32 tiene cómputo suficiente para correr un modelo de síntesis de voz neuronal. La decisión preliminar es separar el reconocimiento del gesto de la síntesis de voz:

- **Canal principal:** el ESP32 envía por Bluetooth el texto o letra reconocida a una app en el celular, y el motor de texto-a-voz nativo del teléfono (Android TextToSpeech/iOS AVSpeechSynthesizer) genera el audio. No requiere servidor, no tiene costo y el vocabulario no está limitado.
- Canal de respaldo sin celular: módulo para el microcontrolador “DFPlayer Mini” con clips de audio pregrabados para un vocabulario inicial reducido (letras y palabras frecuentes), reproducidos por comando serial desde el ESP32. Funciona sin conexión a internet ni Bluetooth.
- Se descarta, para esta etapa, un servidor propio de TTS en la nube: agrega una dependencia de infraestructura que no se justifica frente a usar el motor de voz que ya trae el teléfono.

2.1.2. Modelo de reconocimiento en el microcontrolador

La decisión preliminar es que el modelo de clasificación de gestos corra localmente en el ESP32, sin solicitudes a un servidor externo. La entrada es de baja dimensión (5 valores de flex, más hasta 6 ejes del IMU), por lo que un modelo compacto es suficiente y evita la latencia y la dependencia de red que implicaría consultar un servidor.

- Tipo de modelo: clasificador ligero (kNN o un MLP, discutido en la sección 5.3), entrenado fuera del ESP32 y exportado para inferencia embebida, con TensorFlow Lite Micro (preliminar).
- Las instrucciones vectoriales del ESP32-S3 (LX7) solo aportan una ventaja real si el modelo evoluciona a algo temporal (p. ej. un LSTM/TCN para señas dinámicas); para un clasificador estático como el de esta etapa, no es un factor decisivo entre las tres placas.
- El ruido del ADC del ESP32 original hace necesario calibrar o promediar las lecturas antes de alimentar el modelo; esto se retoma en la sección 5.1 como parte del preprocesamiento.

3. Diagrama de conexiones

El siguiente esquema resume la arquitectura general: los cinco sensores flex y el IMU opcional alimentan al ESP32, que procesa las señales y controla la salida (voz, Bluetooth o pantalla).

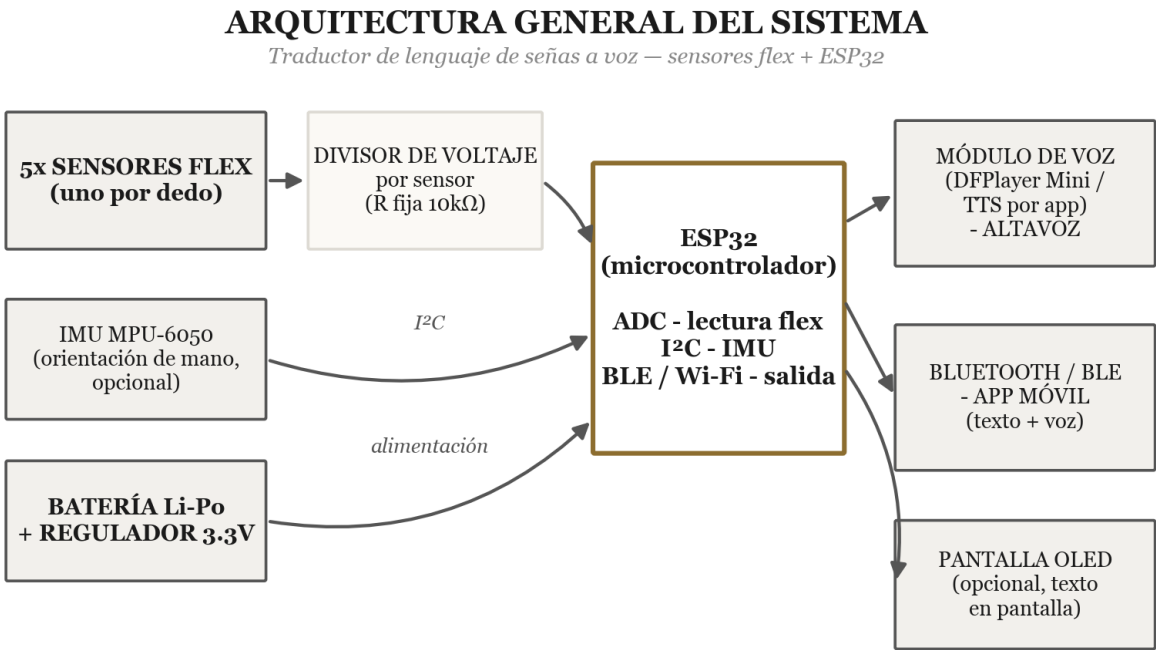


Figura 1. Arquitectura general del sistema.

3.1 Conexión individual de cada sensor flex

Cada sensor flex resistivo se conecta como divisor de voltaje junto a una resistencia fija (recomendado 10 kΩ, ajustable según el rango del sensor usado), y el punto medio se lee por una entrada ADC del ESP32. Este esquema se repite de forma idéntica para los cinco dedos, cambiando únicamente el pin GPIO de lectura.

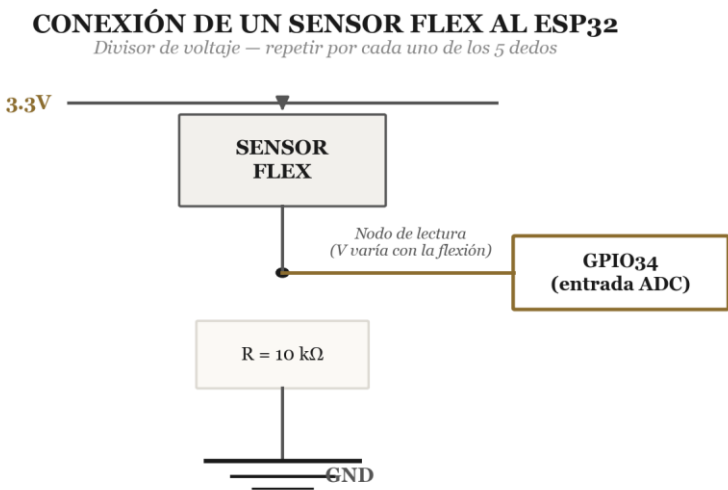


Figura 2. Divisor de voltaje para un sensor flex (repetir x5).

4. Propuestas de implementación física en el guante

Se plantean tres formas de integrar la electrónica en el guante, según la variante de ESP32 elegida en la sección 2.

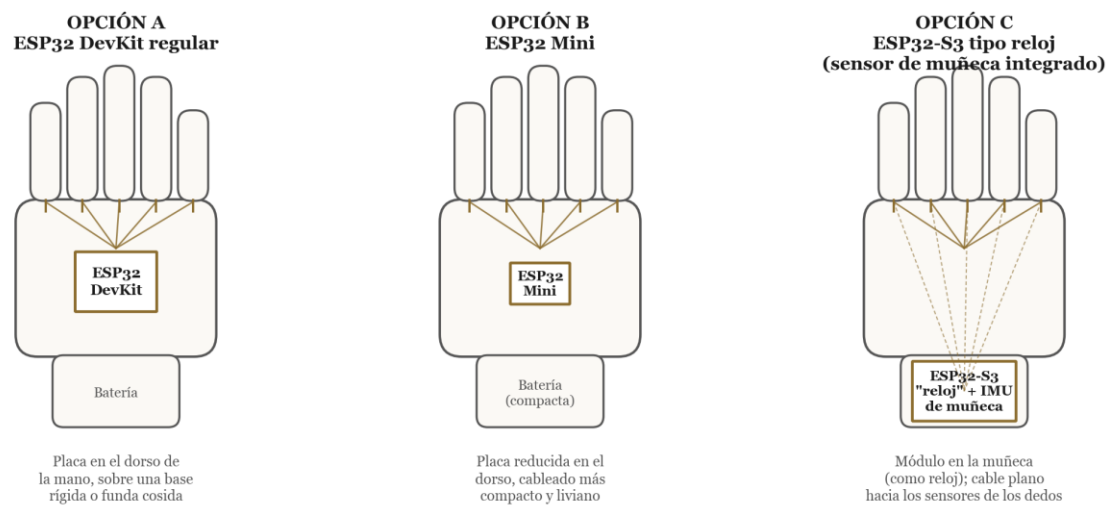


Figura 3. Tres opciones de montaje físico evaluadas.

	Opción A ESP32 DevKit	Opción B ESP32 Mini	Opción C ESP32-S3 reloj
Ventajas	Fácil de programar y depurar; abundante documentación	Más liviana y discreta que la opción A	Aporta IMU de muñeca sin cableado extra; pantalla propia
Desventajas	Bulto notorio en el dorso de la mano	Menos pines disponibles para sensores/IMU	Menos pines libres; requiere cable plano a los dedos; mayor costo
Riesgo técnico	Bajo	Medio (verificar canales ADC suficientes)	Alto (firmware y pinout específicos del modelo)

Tabla 2. Comparación de las tres propuestas de montaje.

5. De los sensores al modelo: pipeline de reconocimiento

Esta sección define, de forma preliminar, cómo se procesan las lecturas de los sensores hasta obtener una letra o gesto reconocido. Es el detalle técnico del bloque “ADC - lectura flex” de la Figura 1, y se ajustará con los datos reales una vez armado el guante (semanas 6+ del cronograma).

5.1. Adquisición y preprocesamiento

- Muestreo: lectura periódica de los 5 canales ADC (y de los ejes del IMU si se incluye) a una frecuencia fija de referencia, en el orden de 50-100 Hz, esto para asegurar el flujo simétrico y correcto de los datos leídos.
- Filtrado: media recortada o mediana sobre un pequeño número de muestras por canal, para atenuar el ruido propio del ADC del ESP32.
- Calibración por sensor: cada flex sensor tiene un rango de voltaje distinto en reposo y en flexión máxima. Se normaliza cada canal a una escala 0-1 usando un mínimo y un máximo obtenidos con una rutina corta de calibración (p. ej. “mano abierta” / “puño cerrado”) que se repite por usuario, ya que el ajuste del guante varía de una mano a otra.

5.2. Ventana de decisión

Para gestos estáticos (deletreo con una posición fija de la mano), el vector de entrada al modelo son los valores normalizados de los 5-11 canales en un instante dado, confirmados sobre una ventana corta (del orden de 200-300 ms) antes de emitir una predicción. Esto evita que se clasifiquen como gesto válido las posiciones intermedias durante la transición de una letra a otra: solo se dispara la clasificación cuando la lectura se mantiene estable dentro de una tolerancia durante ese periodo.

5.3. Modelo de clasificación

- Modelo preliminar: clasificador ligero (kNN sobre los vectores de los datos recolectados por los sensores normalizados, o un MLP pequeño de una capa oculta de unos 16 perceptrones), suficiente para una entrada de baja dimensión como esta.
- kNN: Utiliza la técnica llamada “lazy learning”, por lo que almacena todos los ejemplos etiquetados, y toda la carga de cómputo se pospone hasta el momento de predecir, la consecuencia directa para el ESP32 es que el procesamiento se vuelve mayor al tener más cantidad de datos etiquetados.
- MPL: Es una capa de perceptrones que sí almacenan una matriz de pesos y su razonamiento tiene persistencia una vez entrado. El cómputo fuerte estaría implicado en el entrenamiento en un entorno externo, mientras que el ESP32 se limitaría a usar el modelo solo para predicciones.
- Entrenamiento fuera del ESP32, con los ejemplos etiquetados recolectados del propio guante; exportación cuantizada (INT8) para inferencia embebida con TensorFlow Lite Micro o una herramienta equivalente como Edge Impulse.
- Salida: índice de clase que se traduce a letra/gesto mediante una tabla fija, y ese resultado es lo que se envía al bloque de salida de voz descrito en la sección 2.

5.4. Datos de entrenamiento (preliminar)

Se requerirá recolectar varias repeticiones de cada gesto del vocabulario inicial, idealmente con más de una persona probando el guante, para no sobreajustar el modelo a una sola mano. El proceso concreto de recolección y entrenamiento se detallará en el Documento N.º 002, una vez el hardware esté armado para capturar datos reales.

6. Lista de materiales preliminar

Componente	Cantidad	Notas
ESP32 (variante a definir)	1	Según decisión final de la sección 2
Sensor flex 2.2" (o similar)	5	Uno por dedo
Resistencia 10 k Ω	5	Solo si el sensor no trae divisor incorporado
IMU MPU-6050	1	Opcional; no necesario si se usa la opción C
Módulo DFPlayer Mini + micro SD + mini altavoz	1 set	Para salida de voz local (alternativa a app por Bluetooth)
Batería LiPo 3.7 V, 500-1000 mAh	1	Ajustar según autonomía requerida
Módulo cargador/protección TP4056	1	Carga segura de la batería
Guante base (tela elástica)	1	Soporte físico de sensores y cableado
Cable flexible / cinta plana	-	Conexión sensores-ESP32, especialmente en opción C
Protoboard mini o PCB perforada	1	Para el divisor de voltaje de cada sensor

Tabla 3. Materiales estimados; cantidades y modelos exactos por confirmar.

7. Próximos pasos

Semana	Actividad
Semana 5 (actual)	Definición de arquitectura y comparación de placas ESP32 — este documento
Semana 6	Integración de los 5 sensores; lectura simultánea y mapeo inicial con kNN
Semana 7	Desarrollo de modelo MLP con múltiples capas de perceptrones
Semana 8	Prototipo de salida (voz local o app) y primeras pruebas con el guante armado
Semana 9	Ajustes de ergonomía y elección final de plataforma ESP32 según resultados
Semanas 10+	Documentación de resultados — Informe técnico